

University of Mumbai

PRACTICAL JOURNAL



602 Machine Learning

Journal

SUBMITTED IN PARTIAL FULFILLMENT OF THE
REQUIREMENTS FOR QUALIFYING

Master of Science (IT-Artificial Intelligence)

PART-II - SEMESTER – III

Submitted by

Name: Muhammad Tarique

Seat No: _____

Department of Information Technology

2nd floor, IT park

University of Mumbai, Vidyanagari,

Santacruz (East), Mumbai 400098

2025-26

University of Mumbai



Department of Information Technology

Certificate

This is to certify that Mr./Ms. **Muhammad Tarique**,
Seat No. _____ Studying in **MSC-(IT-AI) Part II**
Semester III has satisfactorily completed the Practical of
602-Machine Learning Journal as prescribed by University of Mumbai, during the
academic year **2025 - 26**.

Signature

Subject-In-Charge

Signature

Head of the Department

Signature External

Examiner

College Seal: _____

Date: _____

INDEX

Sr. No.	List of Practical	Date	Page No.	Sign
1	Perform Data Preprocessing Techniques including Handling Missing Data, Encoding, Normalization, and Feature Scaling.		1 - 6	
2	Implementation of Linear Regression with performance metrics.		7- 10	
3	Implementation of Logistic Regression and evaluate using confusion matrix.		11-13	
4	Implementation of K-Nearest Neighbours algorithm with performance metrics		14-18	
5	Implementation of Support Vector Machine with evaluation using Accuracy, Precision and recall.		19-23	
6	Implement Decision Tree Classifier and do comparative analysis with Random Forest Classifier		24-26	
7	Implementation of Polynomial Regression		27-29	
8	Implementation of and visualize K-Means clustering algorithm.		30-32	
9	Implementation of Hierarchical clustering and visualize using dendrogram.		33-36	
10	Perform Time Series Decomposition and visualization with trend, seasonality, residuals.		37-41	
11	Implementation of Feedforward Neural Network for Handwritten Digit Recognition using MNIST Dataset.		42-45	
12	Implementation of Bayesian Optimization		46-48	

Practical 1

AIM: Perform Data Preprocessing Techniques including Handling Missing Data, Encoding, Normalization, and Feature Scaling.

Theory:

Data preprocessing is an essential step in Machine Learning used to transform raw data into a clean and suitable format for model building. Real-world datasets often contain noise, missing values, inconsistent formatting, and variations in scale. If not handled properly, these issues can lead to inaccurate model predictions. Preprocessing improves the quality and reliability of the dataset.

1) Handling Missing Data

Missing values occur due to incomplete surveys, human errors, system failures, or data corruption.

Machine learning algorithms cannot process missing records effectively.

Common approaches to handle missing data:

Deletion: Removing rows or columns with too many missing values.

Imputation:

- Mean/Median imputation for numerical features.
- Mode imputation for categorical features.

2) Encoding Categorical Data

Many ML algorithms work only with numerical data. Therefore, categorical features must be converted into numbers.

Common encoding methods:

- Label Encoding: Assigns numeric values to categories.
- One-Hot Encoding: Creates binary (0/1) columns for each category.
- Ordinal Encoding: Used when categories have a natural order (e.g., Good > Average > Poor).

Encoding ensures machines can interpret categorical information correctly.

3) Feature Scaling / Normalization

Features measuring different units and ranges can dominate the results. Scaling ensures all features contribute equally to the model.

Two widely used methods:

- Standardization (Z-score scaling)
Converts data to mean = 0 and standard deviation = 1.
- Min-Max Normalization
Converts values to a fixed range, usually 0 to 1.

Code and Output:**1) Data Exploration**

```
import pandas as pd
import numpy as np
data= { 'Name': [ 'Ammar', 'hafeez', 'hurara ', 'wasim'],
        'Age': [21, 30, 24, np.nan],
        'Marks': [ np.nan, 90, 90, 90],
        'City': [ 'Mumbai', 'Pune', 'Bangalore', np.nan] }
df= pd.DataFrame(data)
df.head()
```

	Name	Age	Marks	City
0	Ammar	21.0	NaN	Mumbai
1	hafeez	30.0	90.0	Pune
2	hurara	24.0	90.0	Bangalore
3	wasim	NaN	90.0	NaN

```
df_drop=df.dropna()
df_drop.head()
```

	Name	Age	Marks	City
1	hafeez	30.0	90.0	Pune
2	hurara	24.0	90.0	Bangalore

```
df_drop_col=df.dropna(axis=1)
df_drop_col.head()
```

	Name
0	Ammar
1	hafeez
2	hurara
3	wasim

```
df['Age']=df['Age'].fillna(df['Age'].mean())
df['Marks']=df['Marks'].fillna(df['Marks'].median())
df['City']=df['City'].fillna(df['City'].mode())
df.head()
```

	Name	Age	Marks	City
0	Ammar	21.0	90.0	Mumbai
1	hafeez	30.0	90.0	Pune
2	hurara	24.0	90.0	Bangalore
3	wasim	25.0	90.0	NaN

```
df_ffill=df.fillna(method='ffill')
```

```
df_ffill.head()
```

```
/tmp/ipython-input-2735089857.py:1: FutureWarning:
df_ffill=df.fillna(method='ffill')
```

	Name	Age	Marks	City
0	Ammar	21.0	90.0	Mumbai
1	hafeez	30.0	90.0	Pune
2	hurara	24.0	90.0	Bangalore
3	wasim	25.0	90.0	Bangalore

```
df_bfill['Age'] = df_bfill['Age'].interpolate().ffill()
df_bfill['Marks'] = df_bfill['Marks'].interpolate().ffill()
df_bfill['City'] = df_bfill['City'].ffill().bfill() # fixed: no interpolate
for text
df_bfill.head(10)
```

...

	Name	Age	Marks	City
0	Ammar	21.0	90.0	Mumbai
1	hafeez	30.0	90.0	Pune
2	hurara	24.0	90.0	Bangalore
3	wasim	25.0	90.0	Bangalore

2) Encoding Categorical Data

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import
    MinMaxScaler, StandardScaler, RobustScaler, MaxAbsScaler
df=pd.read_csv('diabetes.csv')
df.head()
```

...

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	12	197	82	3	291	43.1	2.213	72	1
1	5	102	117	44	214	34.0	0.853	43	1
2	0	101	61	14	189	19.6	2.398	38	1
3	3	184	113	32	197	40.4	1.232	51	0
4	11	98	40	4	215	49.8	1.808	55	0

```
min_scaler=MinMaxScaler()
standard_scaler=StandardScaler()
rob_scaler=RobustScaler()
mabs_scaler=MaxAbsScaler()

min_df=min_scaler.fit_transform(df)
standard_df=standard_scaler.fit_transform(df)
rob_df=rob_scaler.fit_transform(df)
mabs_df=mabs_scaler.fit_transform(df)

print(min_df)
print(standard_df
) print(rob_df)
print(mabs_df)
print(" Practical 1 Done by Muhammad tarique ")
```

```

[0.      0.46969697 0.85470085 0.80851064 0.2962963  0.92771084
 1.      0.84615385 1.      ]
[0.14285714 0.94444444 1.      1.      0.11111111 0.94176707
 0.72623883 0.83333333 1.      ]
[0.21428571 0.78282828 0.5982906  0.61702128 0.44781145 0.77710843
 0.21770918 0.35897436 1.      ]
[0.57142857 0.5959596  0.54700855 0.0212766  0.78114478 0.57028112
 0.54021121 0.80769231 1.      ]
[0.07142857 0.6010101  0.35897436 0.65957447 0.85858586 0.69477912
 0.60641755 0.74358974 1.      ]
[0.21428571 0.7020202  0.36752137 0.93617021 0.12121212 0.36144578
 0.80828595 0.61538462 1.      ]
[0.92857143 0.83333333 0.45299145 0.5106383  0.86195286 0.562249
 0.6320065  0.57692308 1.      ]
[0.21428571 0.82828283 0.68376068 0.5106383  0.97643098 0.63453815
 0.25345248 0.78205128 0.      ]
[0.21428571 0.35353535 0.95726496 0.06382979 0.66329966 0.92971888
 0.48415922 0.62820513 0.      ]
[1.      0.92424242 0.5042735  0.38297872 0.85521886 0.79919679
 0.89723802 0.48717949 0.      ]
[0.5      0.53535354 0.95726496 1.      0.26936027 0.65461847
 0.52030869 0.30769231 0.      ]
[0.      0.5959596  0.56410256 0.06382979 0.45791246 0.67269076
 0.73801787 0.34615385 1.      ]
[0.07142857 0.82323232 0.90598291 0.89361702 0.63636364 0.86746988
 0.51462226 1.      1.      ]
[0.64285714 0.84848485 0.78632479 0.25531915 0.43434343 0.50803213
 0.73151909 0.57692308 1.      ]]

```

Practical 1 Done by Muhammad Tarique Ansari 😊

3) Feature Scaling

```

import pandas as pd

from sklearn.preprocessing import LabelEncoder

data = {'color': ['Red', 'Blue', 'Pink', 'Green', 'Pink', 'Blue']}
df = pd.DataFrame(data)

encoder = LabelEncoder()

df['encoded_color'] = encoder.fit_transform(df['color'])

print(df)

```

```

...    color  encoded_color
0     Red             3
1     Blue             0
2     Pink             2
3     Green            1
4     Pink             2
5     Blue             0

```

```

df_onehot=pd.get_dummies(df,columns=['color'])
df_onehot.head()

```

```

...    encoded_color  color_Blue  color_Green  color_Pink  color_Red
0             3         False         False         False         True
1             0          True         False         False         False
2             2         False         False          True         False
3             1         False          True         False         False
4             2         False         False          True         False

```

```
import pandas as pd
```

```
data = {'color': ['Red', 'Blue', 'Pink', 'Green', 'Pink', 'Blue']}
df = pd.DataFrame(data)
```

```
# Frequency encoding
```

```
freq_map = df['color'].value_counts().to_dict()
df['color_Encoded'] = df['color'].map(freq_map)
```

```
print(df)
```

```
print("Practical 1 Done by Muhammad Tarique Ansari")
```

```
***      color  color_Encoded
      0    Red             1
      1   Blue             2
      2   Pink             2
      3  Green             1
      4   Pink             2
      5   Blue             2
Practical 1 Done by Muhammad Tarique Ansari
```

Conclusion:

The practical successfully demonstrated essential data preprocessing techniques such as handling missing values, encoding categorical data, normalization, and feature scaling. These steps improved the quality and consistency of the dataset, making it suitable for accurate analysis and machine-learning model building. Proper preprocessing ensures better performance, reliability, and meaningful outcomes in any data-driven project.

Practical 2

AIM: Implementation of Linear Regression with performance metrics.

Theory:

Linear regression in machine learning is defined as a statistical model that analyzes the linear relationship between a dependent variable and a given set of independent variables. Linear regression is a type of supervised machine-learning algorithm that learns from the labelled datasets and maps the data points with most optimized linear functions which can be used for prediction on new datasets.

Performance Metrics:

1) Mean Absolute Error (MAE)

MAE is the average of the absolute differences between the actual and predicted values.

Formula:
$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}|$$

A smaller MAE indicates that the model predictions are closer to the actual values.

2) Mean Squared Error (MSE)

MSE is the average of the squared differences between actual and predicted values. Formula:

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y})^2$$

MSE penalizes larger errors more than smaller ones due to squaring.

3) R² Score

R² measures the proportion of variance in the dependent variable that is predictable from the independent variables. Formula:

$$R^2 = 1 - \frac{\sum (y_i - \hat{y})^2}{\sum (y_i - \bar{y})^2}$$

Interpretation:

- $R^2 = 1 \rightarrow$ Perfect prediction
- $R^2 = 0 \rightarrow$ Model predicts as well as the mean of target
- $R^2 < 0 \rightarrow$ Model performs worse than predicting the mean

4) Root Mean Squared Error (RMSE)

RMSE is the square root of the mean squared error.

Formula:

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y})^2}$$

Code and Output:

```
# Importing Libraries
import pandas as pd
import numpy as np
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import matplotlib.pyplot as plt

# Load the California Housing Prices dataset
data = fetch_california_housing()
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target

# Split the data into features (X) and target variable (y)
X = df.drop(columns=['target'])
y = df['target']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Create and train the Linear Regression model
model = LinearRegression()
model.fit(X_train, y_train)

# Make predictions on the test set
y_pred = model.predict(X_test)

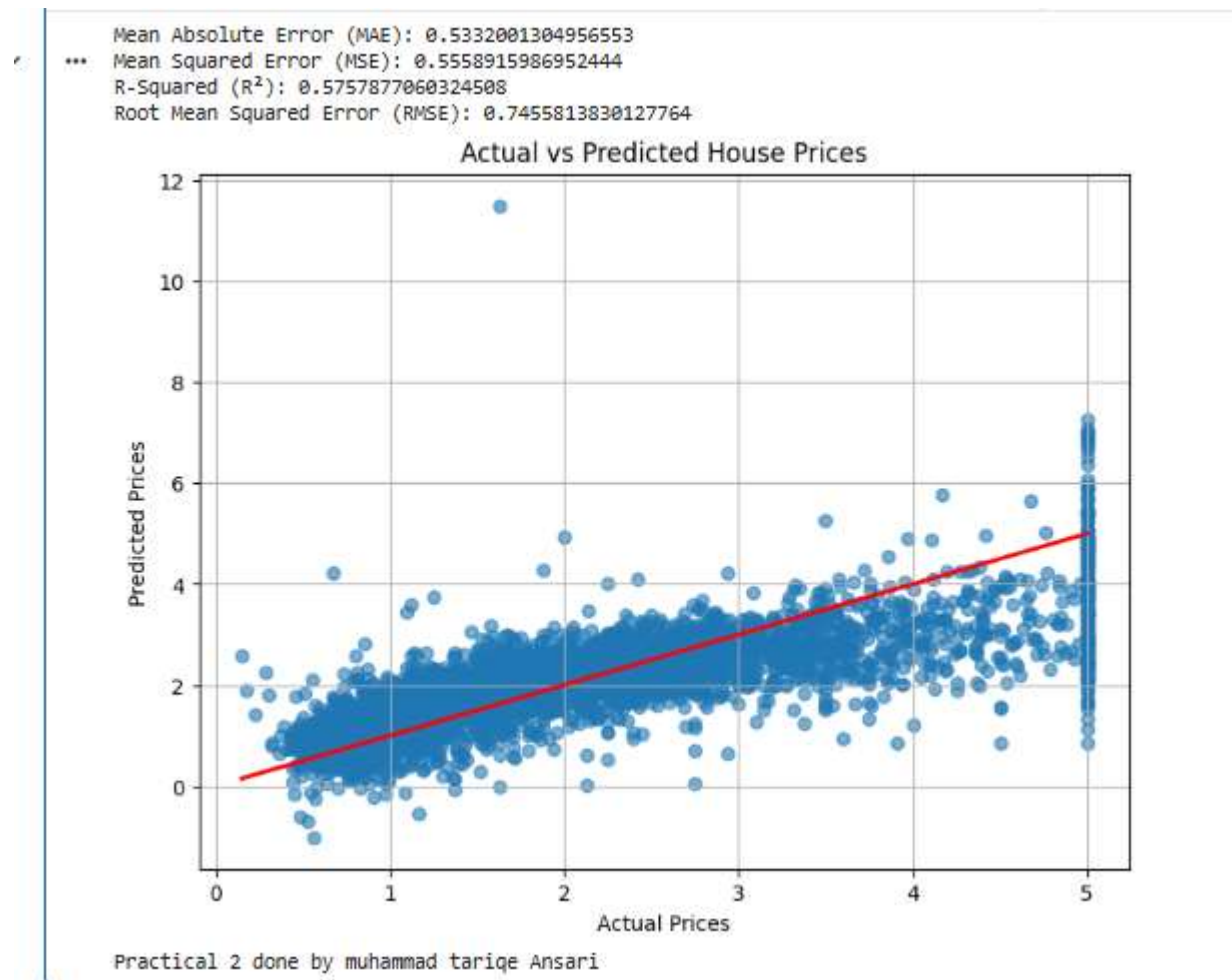
# Calculate evaluation metrics
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
r_squared = r2_score(y_test, y_pred)
rmse = np.sqrt(mse)

# Print the evaluation metrics
print("Mean Absolute Error (MAE):", mae)
print("Mean Squared Error (MSE):", mse)
print("R-Squared (R²):", r_squared)
print("Root Mean Squared Error (RMSE):", rmse)

# Plot actual vs predicted prices
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, alpha=0.6)
plt.plot([y.min(), y.max()], [y.min(), y.max()], color='red', linewidth=2)
plt.xlabel("Actual Prices")
```

```
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted House Prices")
plt.grid(True)
plt.show()

print("Practical 2 done by muhammad tariqe Ansari")
```



```
import numpy as np

y_true = [2, 3, 1, 4, 5.2]
y_pred = [2, 3, 6, 3, 5.0]

# Convert to numpy arrays
y_true = np.array(y_true)
y_pred = np.array(y_pred)

# Difference
d = y_true - y_pred
```

```
# Manual calculations
mae_manual = np.mean(np.abs(d))
mse_manual = np.mean(d**2)
rmse_manual = np.sqrt(mse_manual)

# R² manual
r2_manual = 1 - (np.sum(d**2) / np.sum((y_true - np.mean(y_true))**2))

# Print results
print("Manual MAE:", mae_manual)
print("Manual MSE:", mse_manual)
print("Manual RMSE:", rmse_manual)
print("Manual R²:", r2_manual)

print("Practical 2 done by Muhammad Tariqeu Ansari")
```

```
✓ ... Manual MAE: 1.24
    Manual MSE: 5.208
    Manual RMSE: 2.282104292095346
    Manual R²: -1.4039881831610042
    Practical 2 done by Muhammad Tariqeu Ansari
```

Conclusion:

In this practical, we implemented Linear Regression on the California Housing dataset and evaluated it using MAE, MSE, R^2 and RMSE. The obtained metrics and the Actual vs Predicted plot confirmed that the model was able to learn the linear relationship in the data and make reasonable predictions.

Practical 3

AIM: Implementation of Logistic Regression and evaluate using confusion matrix.

Theory:

Logistic Regression:

Logistic Regression is a supervised machine learning algorithm used for classification problems. Unlike linear regression which predicts continuous values it predicts the probability that an input belongs to a specific class. It is used for binary classification where the output can be one of two possible categories such as Yes/No, True/False or 0/1. It uses sigmoid function to convert inputs into a probability value between 0 and 1.

Evaluation Metrics:

Confusion Matrix:

A table used to visualize the performance of a classification algorithm. It compares the true labels of the dataset to the predictions made by the model.

- True Positive (TP): The model correctly predicted the positive class.
- True Negative (TN): The model correctly predicted the negative class.
- False Positive (FP): The model incorrectly predicted the positive class (Type I error).
- False Negative (FN): The model incorrectly predicted the negative class (Type II error).

Accuracy:

The ratio of correct predictions to the total number of predictions. It provides a general sense of the model's performance.

- Formula: $Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$

Precision:

Measures the accuracy of positive predictions. It is the ratio of true positives to the total number of instances predicted as positive.

- Formula: $Precision = \frac{TP}{TP+FP}$

Also known as sensitivity or the true positive rate (TPR), it measures the proportion of actual positive instances that were correctly identified by the model.

- Formula: $Recall = \frac{TP}{TP+FN}$

Code and Output:

```
import pandas as pd
pima = pd.read_csv("diabetes.csv")
pima.head()
```

...	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	12	197	82	3	291	43.1	2.213	72	1
1	5	102	117	44	214	34.0	0.853	43	1
2	0	101	61	14	189	19.6	2.398	38	1
3	3	184	113	32	197	40.4	1.232	51	0
4	11	98	40	4	215	49.8	1.808	55	0

```
# Split dataset into features and target variable

feature_cols = ['Pregnancies', 'Insulin', 'BMI', 'Age',
                'Glucose', 'BloodPressure', 'DiabetesPedigreeFunction']

X = pima[feature_cols]      # Features
y = pima['Outcome']         # Target variable

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn import metrics

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Initialize model
logreg = LogisticRegression(max_iter=1000)

# Train model
logreg.fit(X_train, y_train)

# Predict on test set
y_pred = logreg.predict(X_test)

# Confusion Matrix
cnf_matrix = metrics.confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cnf_matrix)
```

```
*** Confusion Matrix:  
[[1 5]  
[1 3]]
```

```
print("Accuracy:", metrics.accuracy_score(y_test, y_pred))  
print("Precision:", metrics.precision_score(y_test, y_pred))  
print("Recall:", metrics.recall_score(y_test, y_pred))  
  
print("Practical 3 done by Muhammad Tarique")
```

```
*** Accuracy: 0.4  
Precision: 0.375  
Recall: 0.75  
Practical 3 done by muhammad Tarique Ansari
```

Conclusion:

In this practical, Logistic Regression was used to classify diabetes outcomes, and its performance was evaluated using a confusion matrix and metrics like accuracy, precision and recall, confirming that the model can effectively handle binary classification tasks.

Practical 4

AIM: Implementation of Polynomial Regression.

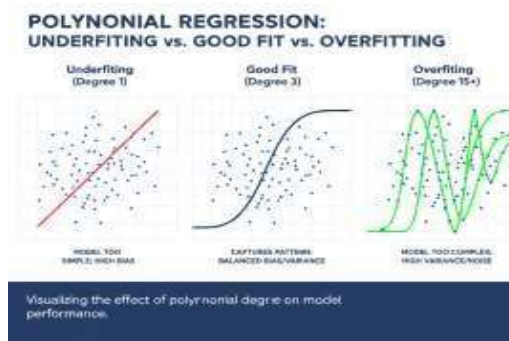
Theory:

Polynomial Regression is a type of regression analysis that models the relationship between the dependent and independent variables as an *n-th degree polynomial*. It is an extension of Linear Regression used when the data shows a **non-linear relationship**. By introducing higher-degree terms of the independent variable, Polynomial Regression can fit curves to data more accurately, making it useful for modeling complex trends.

- It represents the relationship as:

$$Y = b_0 + b_1X + b_2X^2 + \dots + b_nX^n$$

- Captures **non-linear patterns** between variables.
- Uses the **Least Squares Method** to minimize prediction error.
- The **degree of the polynomial** determines model flexibility.
- Evaluated using metrics like **R² score** to measure accuracy.
- Commonly applied where data shows a **curved trend** rather than a straight line.



- **Underfitting (Degree 1):**
 - Model is too simple
 - Misses the real pattern
 - High bias, low accuracy
- **Good Fit (Degree 3):**
 - Model captures the true trend
 - Balanced bias and variance
 - Best performance
- **Overfitting (Degree 15+):**
 - Model is too complex
 - Fits noise instead of pattern
 - High variance, poor test accuracy

Code and Output:

```
# Importing all packages at start
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import r2_score
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures

# Load CSV file
data = pd.read_csv('ice_cream_selling_data.csv')

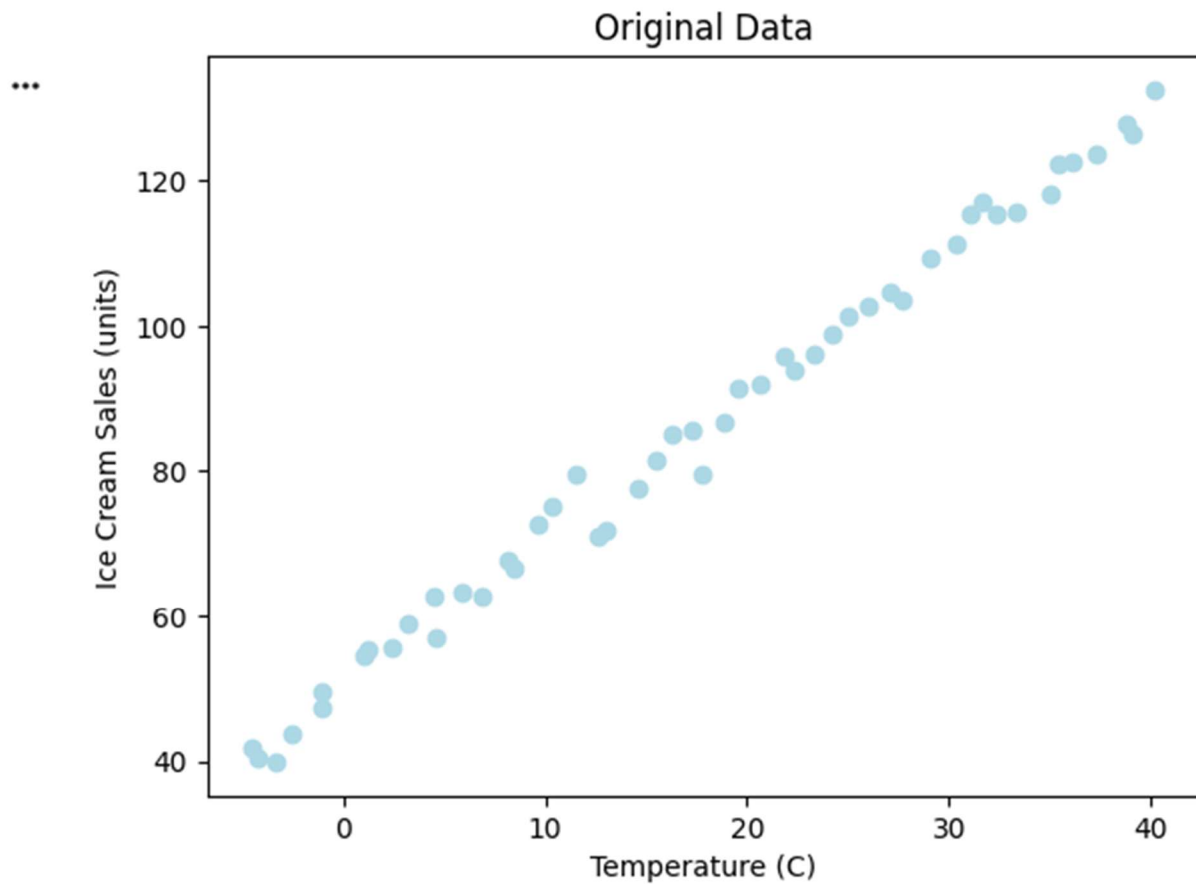
# Display first few rows
print(data.head())
```

...	Temperature (C)	Ice Cream Sales (units)
0	-4.512696	41.875118
1	-4.265160	40.412931
2	-3.321717	39.929012
3	-2.566789	43.818395
4	-1.066908	47.239501

```
# Select features and target
X = data.iloc[:, 0].values.reshape(-1, 1)    # Temperature column
y = data.iloc[:, 1].values                    # Ice Cream Sales column

# Visualize the original data points
plt.scatter(X, y, color="lightblue")        # You can change color

plt.title("Original Data")
plt.xlabel("Temperature (C)")
plt.ylabel("Ice Cream Sales (units)")
plt.show()
```



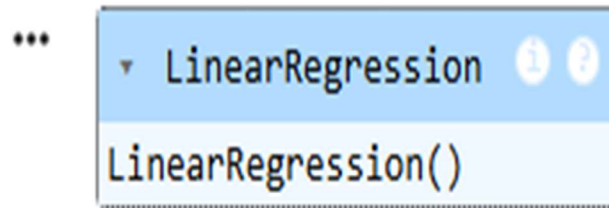
```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

# Create polynomial features (degree 2)
poly_features = PolynomialFeatures(degree=2)

# Transform X into polynomial form
X_poly = poly_features.fit_transform(X)

# Create linear regression model
pr_model = LinearRegression()

# Fit model on polynomial features
pr_model.fit(X_poly, y)
```



```
# Predict values using polynomial regression model
y_pred = pr_model.predict(X_poly)

# Create a dataframe of actual vs predicted values
actual_predicted = pd.DataFrame({
    'Actual Values': y,
    'Predicted Values': y_pred
})

print(actual_predicted)

)
```

...	Actual Values	Predicted Values
0	41.875118	41.280871
1	40.412931	41.784464
2	39.929012	43.702753
3	43.818395	45.236518
4	47.239501	48.280561
5	49.562620	48.216236
6	54.584245	52.536495
7	55.193725	52.873606
8	55.742063	55.386569
9	59.036414	56.897744
10	56.981418	59.788046
11	62.726602	59.509025
12	63.386155	62.411666
13	62.752840	64.223962
14	67.860102	66.987568
15	66.664371	67.486647
16	72.679186	69.890460
17	75.257225	71.308917
18	79.643279	73.703952
19	71.058159	75.868369
20	71.741981	76.695727
21	77.744865	79.879261
22	81.429229	81.567940
23	85.174901	83.161912

```
# Visualize the polynomial regression results
```

```
plt.scatter(X, y, color="green")
```

```
# Plot polynomial regression curve
```

```
plt.plot(X, y_pred, color='red', label='Polynomial Regression (degree=2)')
```

```
plt.xlabel("Temperature (C)")
```

```
plt.ylabel("Ice Cream Sales (units)")
```

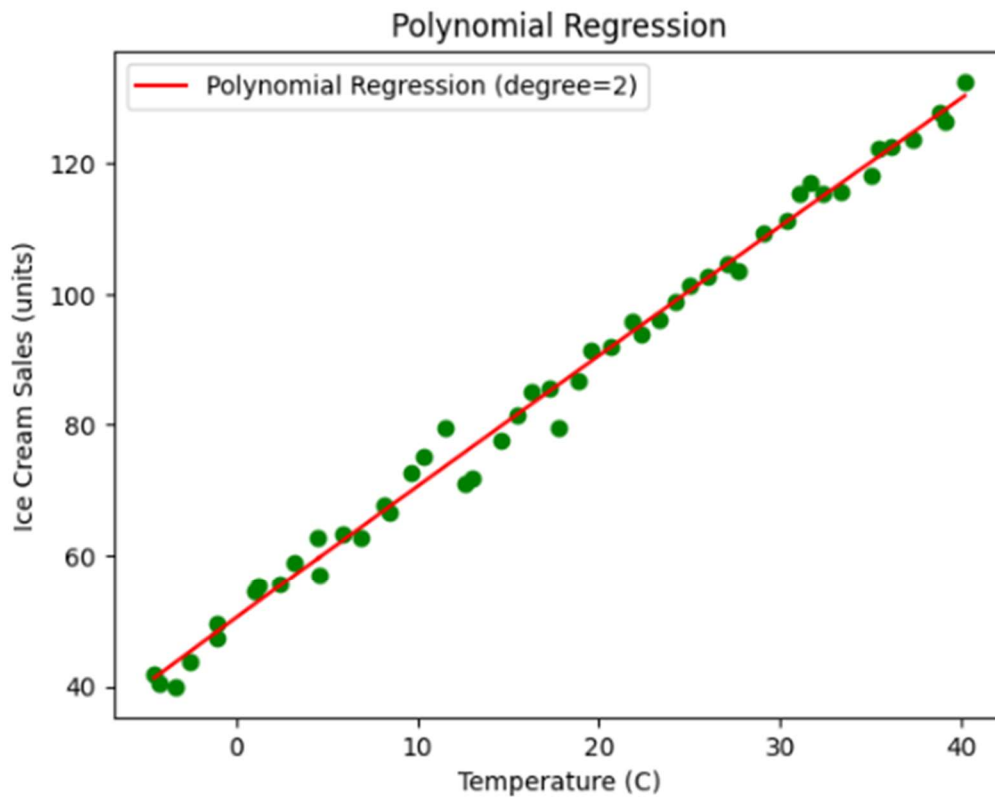
```
plt.legend()
```

```
plt.title('Polynomial Regression')
```

```
plt.show()
```

```
print("Practical 4 done by Muhammad Tarique")
```

...



Practical 4 done by Muhammad Tarique

```
# Predict real value or unseen data
X_new = np.array([[1.9929]]) # Example temperature value to predict

# Transform new input into polynomial form
X_new_poly = poly_features.transform(X_new)

# Predict using the polynomial regression model
y_new_pred = pr_model.predict(X_new_poly)

print("Predicted Ice Cream Sales:", y_new_pred)
```

```
... Predicted Ice Cream Sales: [54.47724234]
```

Conclusion:

Polynomial Regression effectively captured the non-linear relationship between temperature and ice cream sales.

The model provided a good fit, showing that sales increase non-linearly with temperature.

Practical 5

AIM: Implementation of Support Vector Machine with evaluation using Accuracy, Precision and recall.

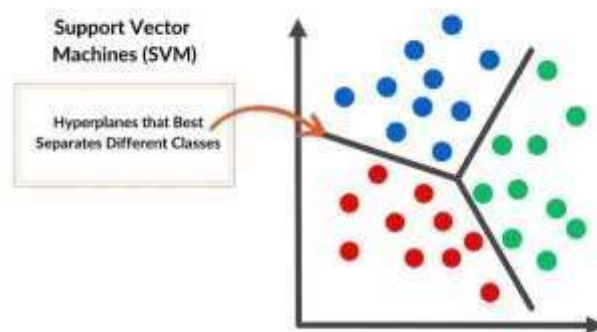
Theory:

SVM offers very high accuracy compared to other classifiers such as logistic regression, and decision trees. It is known for its kernel trick to handle nonlinear input spaces. It is used in a variety of applications such as face detection, intrusion detection, classification of emails, news articles and web pages, classification of genes, and handwriting recognition.

Generally, Support Vector Machines is considered to be a classification approach, it but can be employed in both types of classification and regression problems. It can easily handle multiple continuous and categorical variables. SVM constructs a hyperplane in multidimensional space to separate different classes. SVM generates optimal hyperplane in an iterative manner, which is used to minimize an error. The core idea of SVM is to find a maximum marginal hyperplane (MMH) that best divides the dataset into classes.

SVM Classifiers offer good accuracy and perform faster prediction compared to Naïve Bayes algorithm. They also use less memory because they use a subset of training points in the decision phase. SVM works well with a clear margin of separation and with high dimensional space.

SVM is not suitable for large datasets because of its high training time and it also takes more time in training compared to Naïve Bayes. It works poorly with overlapping classes and is also sensitive to the type of kernel used.



This diagram shows how Support Vector Machines (SVM) classify data by creating hyperplanes (decision boundaries) that best separate different groups of points. Each colored group represents a different class (red, blue, green), and the SVM draws boundaries between them so that each class is clearly separated. The goal is to find the optimal hyperplane that maximizes the margin between classes, giving the best classification results.

Code and Output:

```
# Predict real value or unseen data
X_new = np.array([[1.9929]]) # Example temperature value to predict

# Transform new input into polynomial form
X_new_poly = poly_features.transform(X_new)

# Predict using the polynomial regression model
y_new_pred = pr_model.predict(X_new_poly)

print("Predicted Ice Cream Sales:", y_new_pred)
```

```
... ['mean radius' 'mean texture' 'mean perimeter' 'mean area'
      'mean smoothness' 'mean compactness' 'mean concavity'
      'mean concave points' 'mean symmetry' 'mean fractal dimension'
      'radius error' 'texture error' 'perimeter error' 'area error'
      'smoothness error' 'compactness error' 'concavity error'
      'concave points error' 'symmetry error' 'fractal dimension error'
      'worst radius' 'worst texture' 'worst perimeter' 'worst area'
      'worst smoothness' 'worst compactness' 'worst concavity'
      'worst concave points' 'worst symmetry' 'worst fractal dimension']
```

from

```
sklearn.datasets import load_breast_cancer
```

```
# Load the dataset
data = load_breast_cancer()

# Features and target
X = data.data
y = data.target

# Print the label types (malignant, benign)
print("Labels:", data.target_names)
```

```
Labels: ['malignant' 'benign']
```

```
print("Data (feature) shape:", cancer.data.shape)
```

```
... Data (feature) shape: (569, 30)
```

```
cancer.data.shape
```

(569, 30)

```
print("Top 5 cancer data features:\n", cancer.data[:5])
```

Top 5 cancer data features:

```
[1.799e+01 1.038e+01 1.228e+02 1.001e+03 1.184e-01 2.776e-01 3.001e-01
1.471e-01 2.419e-01 7.871e-02 1.095e+00 9.053e-01 8.589e+00 1.534e+02
6.399e-03 4.904e-02 5.373e-02 1.587e-02 3.003e-02 6.193e-03 2.538e+01
1.733e+01 1.846e+02 2.019e+03 1.622e-01 6.656e-01 7.119e-01 2.654e-01
4.601e-01 1.189e-01]
[2.057e+01 1.777e+01 1.329e+02 1.326e+03 8.474e-02 7.864e-02 8.690e-02
7.017e-02 1.812e-01 5.667e-02 5.435e-01 7.339e-01 3.398e+00 7.408e+01
5.225e-03 1.308e-02 1.860e-02 1.340e-02 1.389e-02 3.532e-03 2.499e+01
2.341e+01 1.588e+02 1.956e+03 1.238e-01 1.866e-01 2.416e-01 1.860e-01
2.750e-01 8.902e-02]
[1.969e+01 2.125e+01 1.300e+02 1.203e+03 1.096e-01 1.599e-01 1.974e-01
1.279e-01 2.069e-01 5.999e-02 7.456e-01 7.869e-01 4.585e+00 9.403e+01
6.150e-03 4.006e-02 3.832e-02 2.058e-02 2.250e-02 4.571e-03 2.357e+01
2.553e+01 1.525e+02 1.709e+03 1.444e-01 4.245e-01 4.504e-01 2.430e-01
3.613e-01 8.758e-02]
[1.142e+01 2.038e+01 7.758e+01 3.861e+02 1.425e-01 2.839e-01 2.414e-01
1.052e-01 2.597e-01 9.744e-02 4.956e-01 1.156e+00 3.445e+00 2.723e+01
9.110e-03 7.458e-02 5.661e-02 1.867e-02 5.963e-02 9.208e-03 1.491e+01
2.650e+01 9.887e+01 5.677e+02 2.098e-01 8.663e-01 6.869e-01 2.575e-01
6.638e-01 1.730e-01]
[2.029e+01 1.434e+01 1.351e+02 1.297e+03 1.003e-01 1.328e-01 1.980e-01
1.043e-01 1.809e-01 5.883e-02 7.572e-01 7.813e-01 5.438e+00 9.444e+01
1.149e-02 2.461e-02 5.688e-02 1.885e-02 1.756e-02 5.115e-03 2.254e+01
1.667e+01 1.522e+02 1.575e+03 1.374e-01 2.050e-01 4.000e-01 1.625e-01
2.364e-01 7.678e-02]]
```

```
print("Target labels (0: malignant, 1: benign):")
```

```
print(cancer.target)
```

Target labels (0: malignant, 1: benign):

[illegible]

```
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_breast_cancer
```

```
from sklearn import svm
from sklearn import metrics

# Load dataset
cancer = load_breast_cancer()

# Split the dataset into training set (70%) and test set (30%)
X_train, X_test, y_train, y_test = train_test_split(
    cancer.data,          # features
    cancer.target,        # labels (0: malignant, 1: benign)
    test_size=0.3,        # 30% for testing
    random_state=109      # ensures reproducibility
)

# Create an SVM classifier with linear kernel
clf = svm.SVC(kernel='linear')

# Train the model on training data
clf.fit(X_train, y_train)

# Predict the response for test dataset
y_pred = clf.predict(X_test)

# Print model accuracy
print("Accuracy:", metrics.accuracy_score(y_test, y_pred))
```

Accuracy: 0.9649122807017544

```
from sklearn import metrics

# Print precision and recall
print("Precision:", metrics.precision_score(y_test, y_pred))
print("Recall:", metrics.recall_score(y_test, y_pred))

print("Practical: 5, Done by : Muhammad Tarique")
```

```
... Precision: 0.9811320754716981
    Recall: 0.9629629629629629
    Practical: 5, Done by : Muhammad Tarique
```

Conclusion:

In this practical, a Support Vector Machine classifier was trained on the breast cancer dataset to separate malignant and benign cases. The model's performance was evaluated using Accuracy, Precision and Recall, which showed that SVM can effectively classify high- dimensional data with a clear margin of separation.

Practical 6

AIM: Implementation of K-Nearest Neighbours Algorithm with Performance Evaluation

Theory:

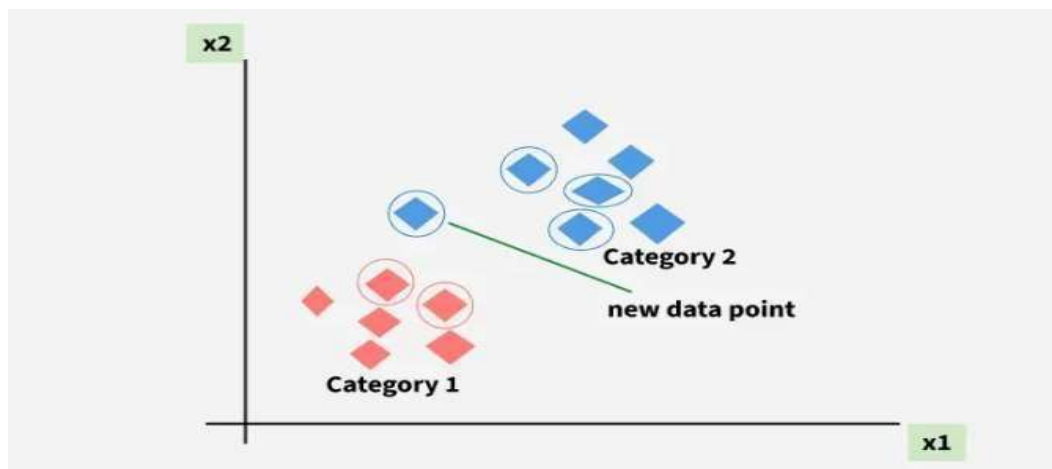
K-Nearest Neighbors (KNN) is a supervised machine learning algorithm generally used for classification but can also be used for regression tasks. It works by finding the "k" closest data points (neighbors) to a given input and makes a predictions based on the majority class (for classification) or the average value (for regression). Since KNN makes no assumptions about the underlying data distribution it makes it a non-parametric and instance-based learning method.

K-Nearest Neighbors is also called as a lazy learner algorithm because it does not learn from the training set immediately instead it stores the entire dataset and performs computations only at the time of classification.

In the k-Nearest Neighbours algorithm k is just a number that tells the algorithm how many nearby points or neighbors to look at when it makes a decision.

Core Concepts

- **Working Principle:** The algorithm works by identifying the "k" closest data points (neighbors) to a specific input. It then makes a prediction based on the majority class (for classification) or the average value (for regression) of these neighbors.
- **Instance-Based Learning:** KNN is considered a "lazy learner" because it does not learn a fixed model from the training data immediately. Instead, it stores the entire dataset and performs computations only when a classification is needed.
- **Non-Parametric:** The algorithm is non-parametric, meaning it makes no underlying assumptions about the distribution of the data



Code and Output:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
dataset = pd.read_csv('Social_Network_Ads.csv')
dataset.head()
```

...	User ID	Gender	Age	EstimatedSalary	Purchased
0	15851400	Male	37	127103	1
1	15600879	Male	36	105043	0
2	15703245	Male	19	83062	0
3	15595816	Male	22	38150	0
4	15937782	Female	58	135695	0

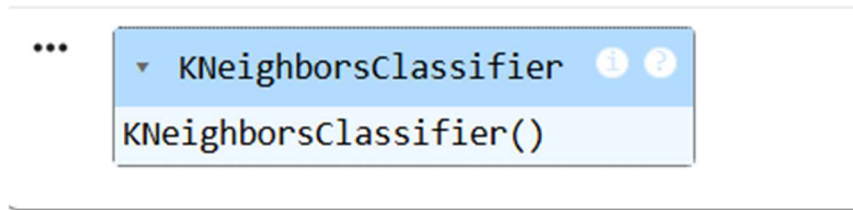
```
# Splitting the dataset into the Training set and Test set
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.20, random_state=0)

# Feature Scaling
from sklearn.preprocessing import StandardScaler

sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

# Training the K-NN model on the Training set from
sklearn.neighbors import KNeighborsClassifier
classifier = KNeighborsClassifier(n_neighbors=5, metric='minkowski',
p=2) classifier.fit(X_train, y_train)
```



```
# Predicting the Test set results

y_pred = classifier.predict(X_test)

from sklearn.metrics import confusion_matrix, accuracy_score

# Confusion Matrix & Accuracy
cm = confusion_matrix(y_test, y_pred)
ac = accuracy_score(y_test, y_pred)

print("Confusion Matrix:\n", cm)
print("Accuracy score is:", ac)

print("Practical no 6: Done by Muhammad Tarique Ansari")
```

```
... Confusion Matrix:
      [[42  5]
       [ 0 67]]
Accuracy score is: 0.956140350877193
Practical no 6: Done by Muhammad Tarique Ansari
```

Conclusion:

In this practical, the K-Nearest Neighbors algorithm was applied to classify user purchase behavior. After training and predicting on the dataset, the model was evaluated using a confusion matrix and accuracy score, which confirmed that KNN can effectively classify data based on similarity to nearby points.

Practical 7

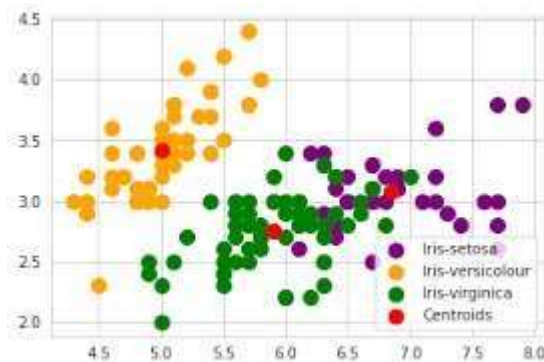
Aim: Implementation and visualization of K-Means Clustering.

Theory:

K-Means Clustering is an unsupervised machine learning algorithm that helps group data points into clusters based on their inherent similarity. Unlike supervised learning, where we train models using labeled data, K-Means is used when we have data that is not labeled and the goal is to uncover hidden patterns or structures. For example, an online store can use KMeans to segment customers into groups like "Budget Shoppers," "Frequent Buyers," and "Big Spenders" based on their purchase history. K-

Means follows an iterative process:

1. Choose the number of clusters (K)
2. Initialize centroids randomly
3. Assign each data point to the nearest centroid
 - Based on distance (usually Euclidean distance)
4. Recalculate centroids by taking the mean of all points in a cluster
5. Repeat step 3 & 4 until centroids no longer change



- The diagram shows how the three types of Iris flowers form natural groups (clusters) based on their measurements.
- The x-axis and y-axis represent two flower measurements (like petal or sepal size). The points of the same color stay close together because they share similar features.
- The **centroids** are the average positions of each group and help identify the center of each cluster.

Code and Output:

```
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.cluster import KMeans
import sklearn.metrics as sm
import pandas as pd
import numpy as np

# Load IRIS dataset
dataset = datasets.load_iris()

print(dataset)
```

```
... {'data': array([[5.1, 3.5, 1.4, 0.2],
                    [4.9, 3. , 1.4, 0.2],
                    [4.7, 3.2, 1.3, 0.2],
                    [4.6, 3.1, 1.5, 0.2],
                    [5. , 3.6, 1.4, 0.2],
                    [5.4, 3.9, 1.7, 0.4],
                    [4.6, 3.4, 1.4, 0.3],
                    [5. , 3.4, 1.5, 0.2],
                    [4.4, 2.9, 1.4, 0.2],
                    [4.9, 3.1, 1.5, 0.1],
                    [5.4, 3.7, 1.5, 0.2],
                    [4.8, 3.4, 1.6, 0.2],
                    [4.8, 3. , 1.4, 0.1],
                    [4.3, 3. , 1.1, 0.1],
                    [5.8, 4. , 1.2, 0.2],
                    [5.7, 4.4, 1.5, 0.4],
                    [5.4, 3.9, 1.3, 0.4],
                    [5.1, 3.5, 1.4, 0.3],
                    [5.7, 3.8, 1.7, 0.3],
                    [5.1, 3.8, 1.5, 0.3],
                    [5.4, 3.4, 1.7, 0.2],
                    [5.1, 3.7, 1.5, 0.4],
                    [4.6, 3.6, 1. , 0.2],
                    [5.1, 3.3, 1.7, 0.5],
```

```
X=pd.DataFrame(dataset.data)
X.columns=['Sepal_Length', 'Sepal_Width', 'Petal_Length', 'Petal_Width']
y=pd.DataFrame(dataset.target)
y.columns=['Targets']
print(X)
```

...	Sepal_Length	Sepal_Width	Petal_Length	Petal_Width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
..	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

[150 rows x 4 columns]

```
import matplotlib.pyplot as plt

from sklearn import datasets
from sklearn.cluster import KMeans
import pandas as pd
import numpy as np

# Load dataset
iris = datasets.load_iris()

# Convert into DataFrame
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.DataFrame(iris.target, columns=['target'])

# Colors
colormap = np.array(['red', 'lime', 'black'])

# Create figure
plt.figure(figsize=(14, 7))

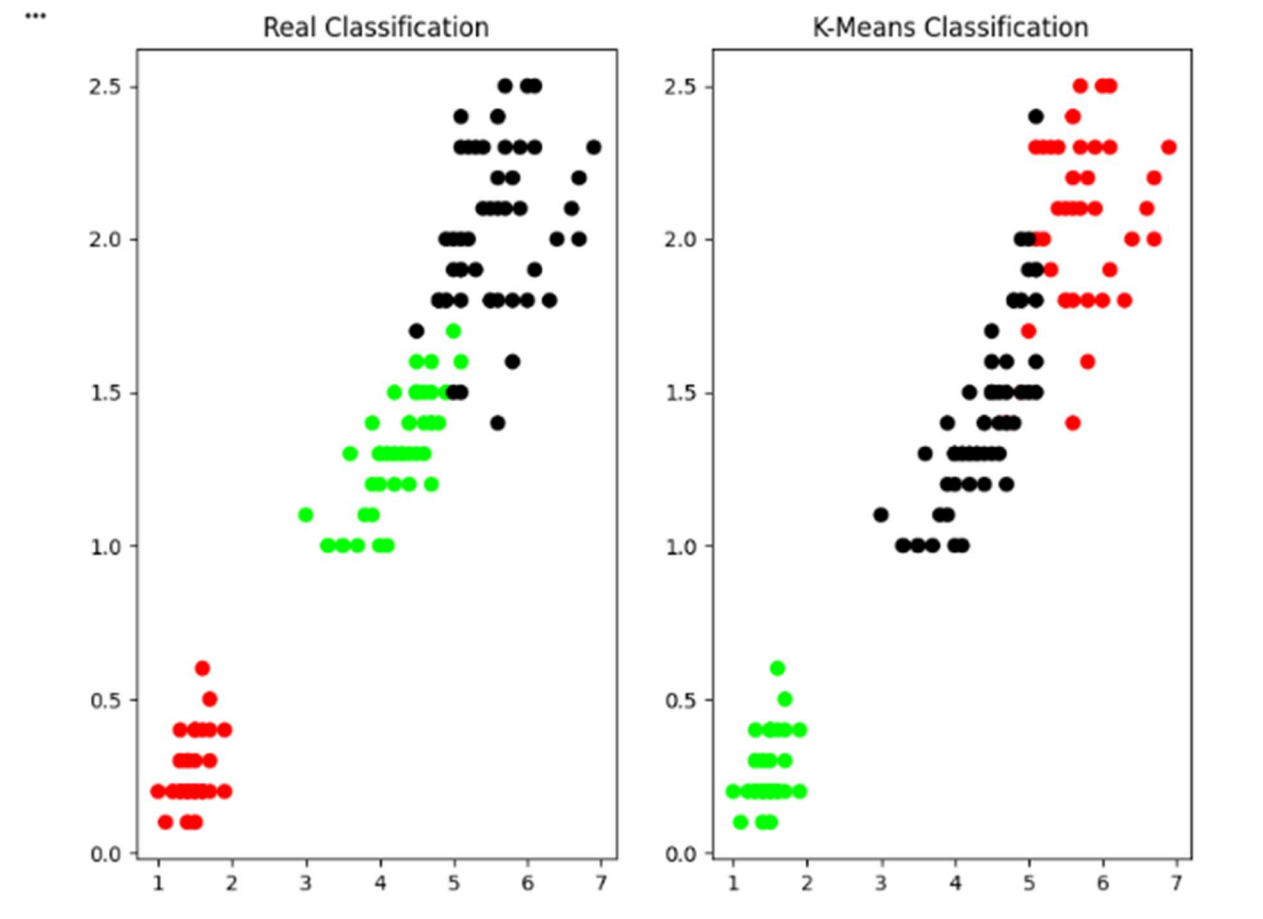
# ----- REAL CLASSIFICATION -----
plt.subplot(1, 3, 1)
plt.scatter(X['petal length (cm)'], X['petal width (cm)'],
            c=colormap[y['target']], s=40)
plt.title('Real Classification')

# ----- K-MEANS CLASSIFICATION -----
plt.subplot(1, 3, 2)
model = KMeans(n_clusters=3, random_state=42)
model.fit(X)

predY = np.choose(model.labels_, [0, 1, 2]).astype(int)
```

```
plt.scatter(X['petal length (cm)'], X['petal width (cm)'],
            c=colormap[predY], s=40)
plt.title('K-Means Classification')

plt.show()
```



```
print('The accuracy score of K-Mean:', sm.accuracy_score(y, model.labels_))
print('The Confusion matrix of K-Mean:\n', sm.confusion_matrix(y,
model.labels_))
print("Practical: 7 Done by, Muhammad Tarique Ansari")
```

```
... The accuracy score of K-Mean: 0.09333333333333334
The Confusion matrix of K-Mean:
[[ 0 50  0]
 [ 3  0 47]
 [36  0 14]]
Practical: 7 Done by, Muhammad Tarique Ansari
```

Conclusion:

In this practical, K-Means clustering was used on the Iris dataset to form groups based on similarity, and the results showed that the algorithm can cluster the data effectively without labeled output.

Practical 8

Aim: Implementation and Visualization of Hierarchical Clustering using Dendrogram.

Theory:

Hierarchical Clustering is a type of unsupervised learning algorithm that builds a hierarchy (tree structure) of clusters instead of forming them directly. It groups data points in a nested sequence — smaller clusters merge into bigger ones (or vice versa) — and the result is shown as a dendrogram (a tree-like diagram).

Types of Hierarchical Clustering

- **Agglomerative Clustering (Bottom-Up):**
 - It uses bottom-up strategy.
 - Starts with each data point as its own cluster.
 - Then, the closest clusters are merged step-by-step until only one big cluster remains.
 - Most commonly used type of hierarchical
- **clustering. Divisive Clustering (Top-Down):**
 - Method employs top-down strategy
 - Starts with one big cluster containing all data points, which is the hierarchy root.
 - Then, it recursively splits the cluster into smaller ones until each data point is alone.
 - In either agglomerative or divisive Hierarchical Clustering, a user can specify the desired number of clusters as a termination condition.

Visualizing with a Dendrogram

1. The result of hierarchical clustering is typically visualized using a **dendrogram**. A dendrogram is a tree-like diagram that records the sequence of merges or splits that occurred and the distance level at which each operation took place.

Interpreting the Dendrogram

1. Horizontal Axis: Represents the data points (or observation labels).
2. Vertical Axis (Height or Distance): Represents the distance or dissimilarity level at which the clusters were merged.
3. Branches: The connections (branches) between data points and groups represent the clusters.
4. Cutting the Tree: To determine the final clusters, you draw a horizontal line ("cut") across the dendrogram at a chosen distance level. The number of vertical lines the cut intersects is the final number of clusters.
 - High Cut: Results in fewer, larger clusters (high inter-cluster distance).
 - Low Cut: Results in many small clusters (low inter-cluster distance).

Code and Ouput:

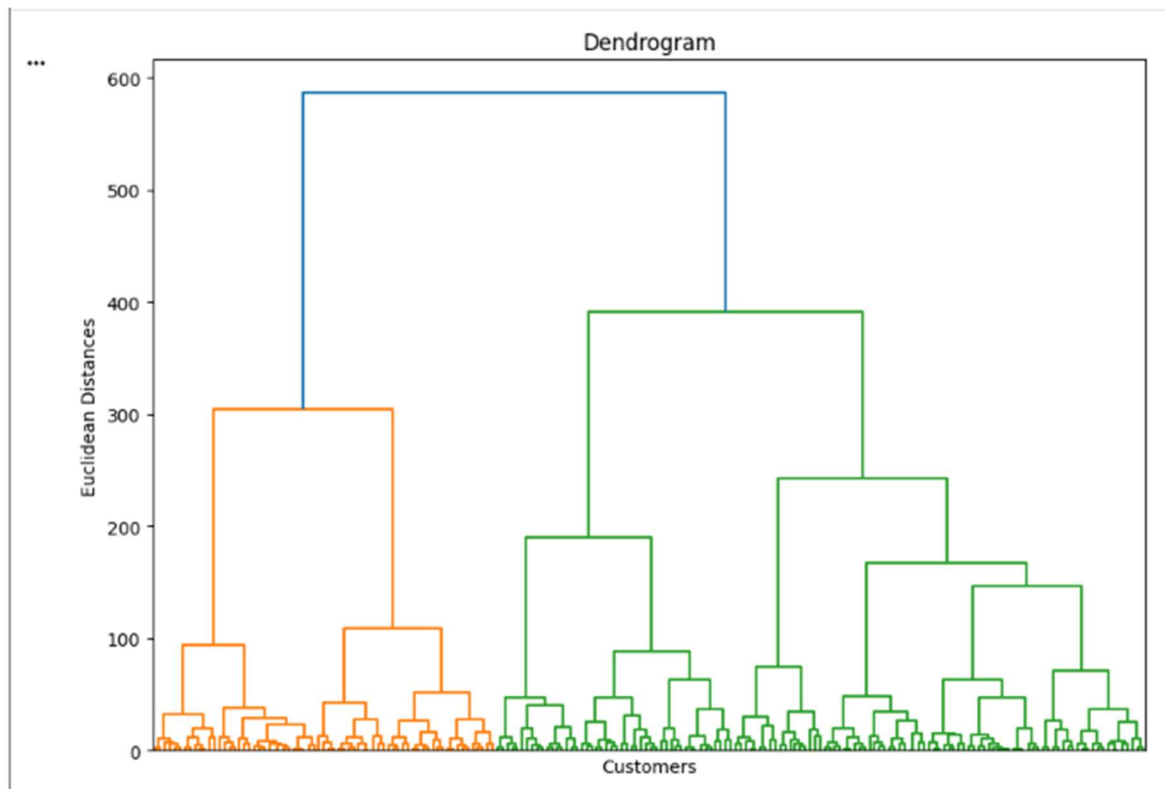
```
# Import libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import scipy.cluster.hierarchy as sch

# Load dataset
dataset = pd.read_csv('Mall_Customers.csv')

# Select only Annual Income and Spending Score columns
X = dataset.iloc[:, [3, 4]].values

# Create Dendrogram
plt.figure(figsize=(10, 7))
dendro = sch.dendrogram(sch.linkage(X, method='ward'), no_labels=True)

plt.title('Dendrogram')
plt.xlabel('Customers')
plt.ylabel('Euclidean Distances')
plt.show()
```



```
#Fitting Agglomerative Hierarchical Clustering to the dataset
```

```
from sklearn.cluster import AgglomerativeClustering
```

```
hc = AgglomerativeClustering(n_clusters = 5, metric = 'euclidean',
linkage = 'ward') y_hc = hc.fit_predict(X)
print(y_hc)
```

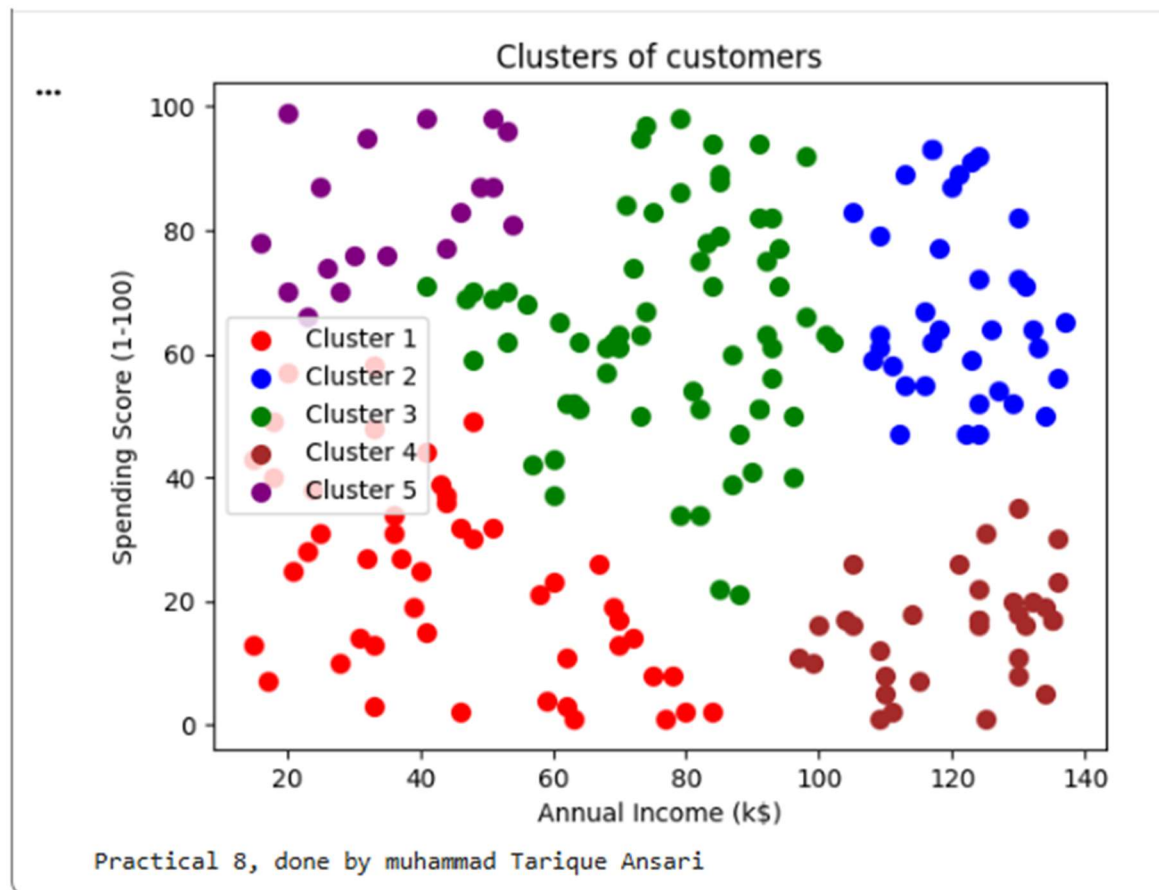
```
... [3 3 2 2 2 1 2 0 2 0 1 4 4 4 0 0 2 1 0 3 0 4 1 0 2 3 2 2 0 2 1 1 2 3 1 4 2
0 2 3 0 0 0 4 0 0 2 2 1 0 2 3 2 2 0 0 0 3 2 0 4 1 4 2 2 1 4 2 2 0 2 3 3 1
1 2 3 0 0 4 2 0 1 2 0 2 3 2 2 0 0 2 2 4 4 1 2 1 0 2 1 0 1 1 2 0 3 3 2 2 1
3 2 0 3 2 1 3 4 0 3 0 0 4 3 0 2 2 3 1 2 1 1 3 4 1 3 2 2 0 2 3 4 2 3 2 0 0
2 0 3 2 2 1 3 2 2 2 1 2 2 2 2 0 3 3 0 4 0 1 3 1 2 3 0 1 0 1 1 4 3 2 2 2 1
3 0 2 1 0 3 1 2 1 1 0 0 2 0 2]
```

```
# Visualise the clusters
```

```
plt.scatter(X[y_hc == 0, 0], X[y_hc == 0, 1], s=50, c='red', label='Cluster
1')
plt.scatter(X[y_hc == 1, 0], X[y_hc == 1, 1], s=50, c='blue', label='Cluster
2')
plt.scatter(X[y_hc == 2, 0], X[y_hc == 2, 1], s=50, c='green', label='Cluster
3')
plt.scatter(X[y_hc == 3, 0], X[y_hc == 3, 1], s=50, c='brown', label='Cluster
4')
plt.scatter(X[y_hc == 4, 0], X[y_hc == 4, 1], s=50, c='purple', label='Cluster
5')
```

```
plt.title('Clusters of customers')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.legend()
plt.show()
```

```
print("Practical 8, done by muhammad Tarique Ansari")
```



Conclusion: Hierarchical clustering was successfully implemented to group customers based on income and spending score. The dendrogram and cluster visualization clearly showed five distinct customer segments.

Practical 9

Aim: Implementation of Density-Based Spatial Clustering of Applications with Noise.

Theory:

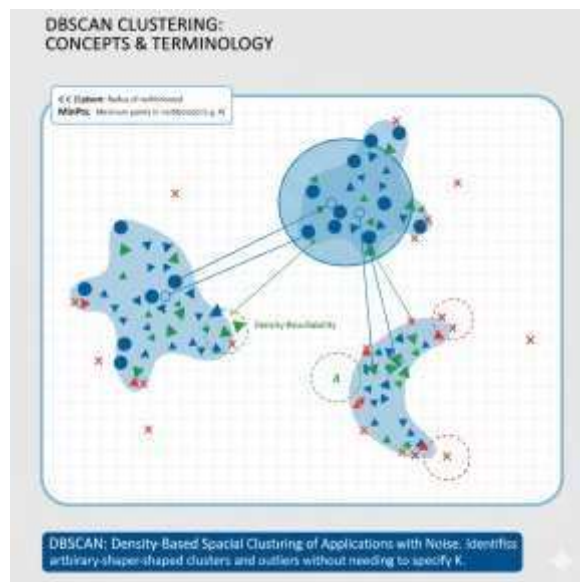
DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a non-hierarchical clustering algorithm that groups together points that are closely packed together, marking as outliers points that lie alone in low-density regions.¹ Its main theoretical advantage over KMeans is that it does not require a predefined number of clusters (² K) and can find clusters of arbitrary shape.

Theoretical Advantages

- Arbitrary Shape Clusters: DBSCAN does not assume spherical clusters (unlike K-Means) and can effectively find clusters with complex shapes, such as crescent moons or nested rings.²⁴
- Noise Handling: It naturally separates noise (outliers) into the "Noise Point" category, which is a major benefit over partition-based methods.²⁵
- No K Required: The user only needs to specify density parameters (²⁶ ϵ and MinPts), which relate more intuitively to the spatial properties of the data than the number of clusters.²⁷

Theoretical Limitations

- Varying Density: DBSCAN struggles with datasets where clusters have significantly varying densities, setting cannot effectively capture both sparse and dense clusters simultaneously.
- High Dimensionality: In high-dimensional spaces, the concept of distance and density becomes less meaningful (the "Curse of Dimensionality"), making it difficult to select.

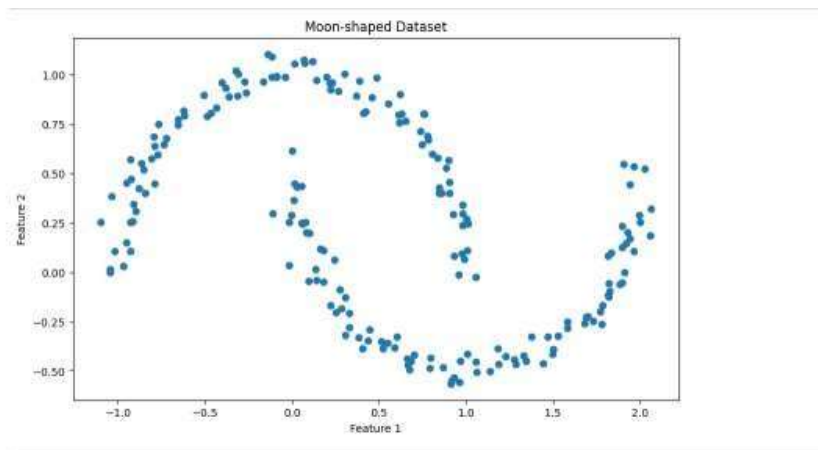


Code and Output:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.cluster import DBSCAN
from sklearn.neighbors import NearestNeighbors

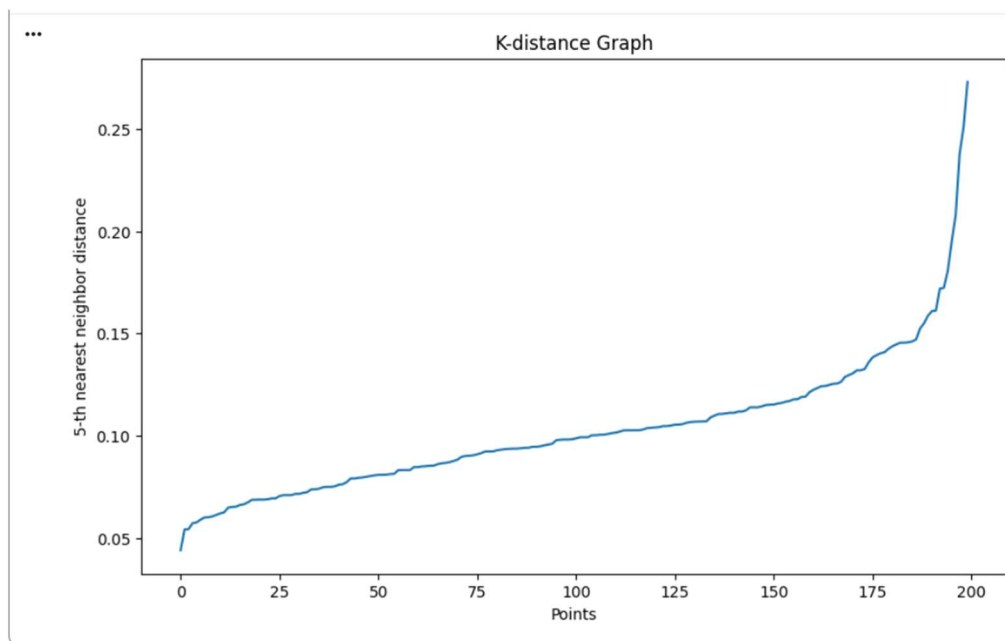
# Generate dataset
X, _ = make_moons(n_samples=200, noise=0.05, random_state=42)

# Visualize the dataset
plt.figure(figsize=(10, 6))
plt.scatter(X[:, 0], X[:, 1])
plt.title('Moon-shaped Dataset')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

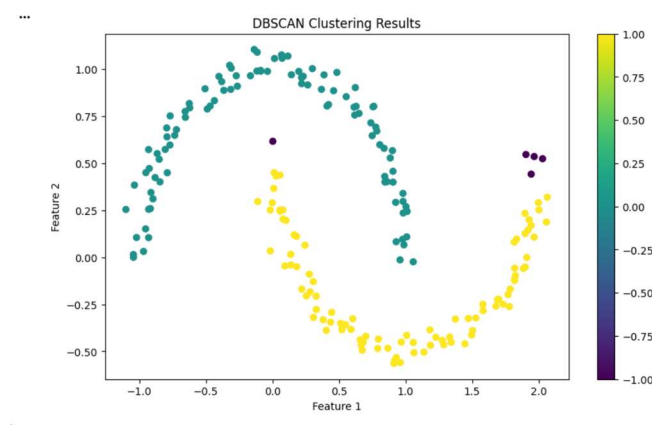


```
# Function to plot k-distance graph
def plot_k_distance_graph(X, k):
    neigh = NearestNeighbors(n_neighbors=k)
    neigh.fit(X)
    distances, _ = neigh.kneighbors(X)
    distances = np.sort(distances[:, k-1])
    plt.figure(figsize=(10, 6))
    plt.plot(distances)
    plt.xlabel('Points')
    plt.ylabel(f'{k}-th nearest neighbor distance')
    plt.title('K-distance Graph')
    plt.show()

# Plot k-distance graph
plot_k_distance_graph(X, k=5)
```



```
# Perform DBSCAN clustering
epsilon = 0.15 # Chosen based on k-distance graph
min_samples = 5 # 2 * num_features (2D data)
dbscan = DBSCAN(eps=epsilon, min_samples=min_samples)
clusters = dbscan.fit_predict(X)
# Visualize the results
plt.figure(figsize=(10, 6))
scatter = plt.scatter(X[:, 0], X[:, 1], c=clusters, cmap='viridis')
plt.colorbar(scatter)
plt.title('DBSCAN Clustering Results')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```



```
# Print number of clusters and noise points

n_clusters = len(set(clusters)) - (1 if -1 in clusters else 0)

n_noise = list(clusters).count(-1)

print(f'Number of clusters: {n_clusters}')

print(f'Number of noise points: {n_noise}')
```

```
Number of clusters: 2
Number of noise points: 5
```

```
print(clusters) print("practical 9
Done by Muhammad Tarique")
```

```
... [ 0 0 0 1 1 1 0 1 1 0 0 1 1 0 1 1 0 1 0 0 1 0 1 0
      0 0 0 0 1 1 0 0 0 0 0 1 1 1 1 0 0 -1 0 0 1 -1 0 0
      1 0 1 1 0 1 1 0 1 0 1 0 1 0 0 0 0 1 0 1 0 1 0 1
      1 0 1 1 0 1 -1 1 0 0 0 1 1 0 1 0 1 0 0 1 1 0 0 1
      1 0 0 1 0 0 1 1 1 1 0 0 1 1 1 0 -1 1 0 1 0 0 1 0
      0 1 1 1 0 0 1 1 0 0 1 0 0 1 0 1 0 1 0 1 1 1 1 0
      0 0 0 1 0 0 1 1 0 0 0 0 1 1 0 0 1 1 1 1 0 0 1 0
      1 0 0 0 0 0 1 1 1 0 1 1 0 0 1 1 1 1 0 1 0 0 1 -1
      0 1 0 1 0 0 1 1]
```

practical 9 Done by Muhammad Tarique Ansari

Conclusion:

DBSCAN was successfully implemented on the moon-shaped dataset, effectively identifying clusters of arbitrary, non-spherical shapes. Using the defined density parameters, the algorithm successfully isolated points as noise (outliers).

The analysis resulted in the identification of distinct, well-defined clusters by density connectivity, validating the clustering process. This confirms DBSCAN's effectiveness for datasets with complex geometry and its advantage in handling noise without needing to prespecify the number of clusters.

Practical 10

Aim: Comparative Analysis of Decision Tree Classifier and Random Forest Classifier

Theory:

1) Decision Tree Classifier

A Decision Tree is a flow-chart-like structure where each internal node represents a feature, each branch represents a decision rule, and each leaf node represents an outcome (class label). It splits the dataset based on the feature that provides the maximum information gain or minimum impurity.

Entropy: Measures disorder or impurity in data.

Information Gain: Improvement in purity after a split.

Gini Index: Impurity metric used for splitting.

Advantages

- Easy to understand and visualize
- Handles both numerical and categorical data
- Prone to overfitting
- Highly sensitive to changes in data

Limitations

2) Random Forest Classifier

Random Forest is an ensemble learning method that builds multiple Decision Trees and combines their results. It uses bagging (Bootstrap Aggregating) and random feature selection to create diverse trees.

It works by creating several decision trees on random subsets of data and features and aggregates their predictions through majority voting.

This reduces variance and improves prediction accuracy.

Advantages

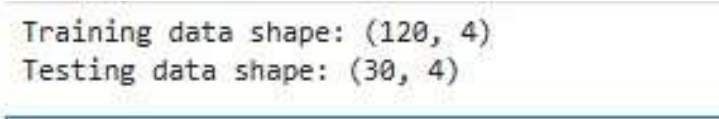
- High accuracy and robustness
- Handles noise and overfitting better than a single Decision Tree
- More complex and computationally expensive
- Harder to interpret compared to a single tree

Limitations

Code and output:

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection
import train_test_split
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
# Load the Iris Dataset
iris = load_iris()
X = iris.data
y = iris.target

from sklearn.model_selection
import train_test_split
# split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print('Training data shape:', X_train.shape)
print('Testing data shape:', X_test.shape)
```



```
Training data shape: (120, 4)
Testing data shape: (30, 4)
```

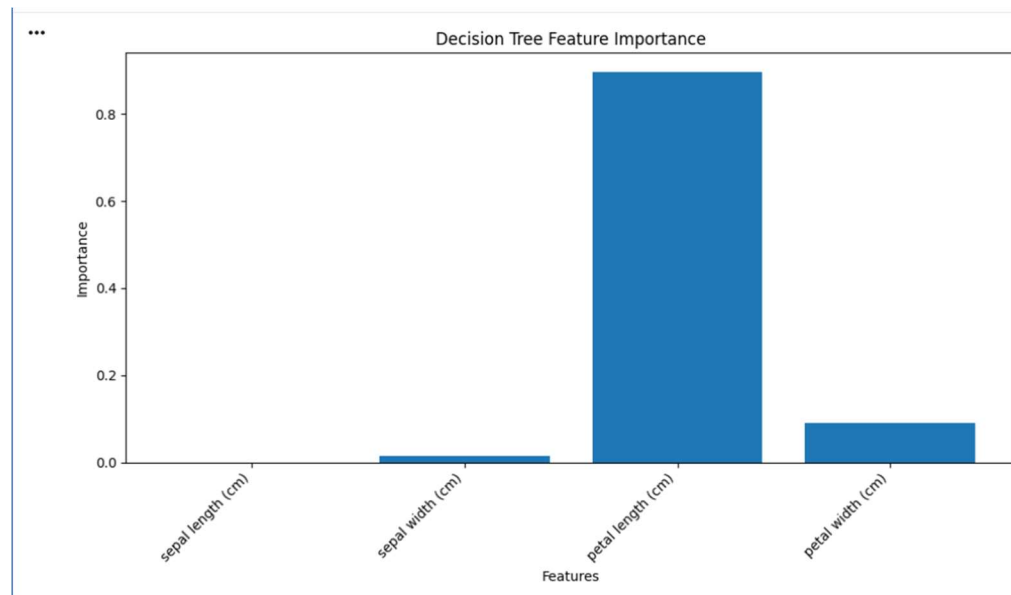
```
# Initiate a Decision Tree classifier and fit it to the training data
dt_clf = DecisionTreeClassifier(criterion="entropy", random_state=42)
dt_clf.fit(X_train, y_train)

# Access the feature importances
#you can skip this for journal
feature_importances = dt_clf.feature_importances_

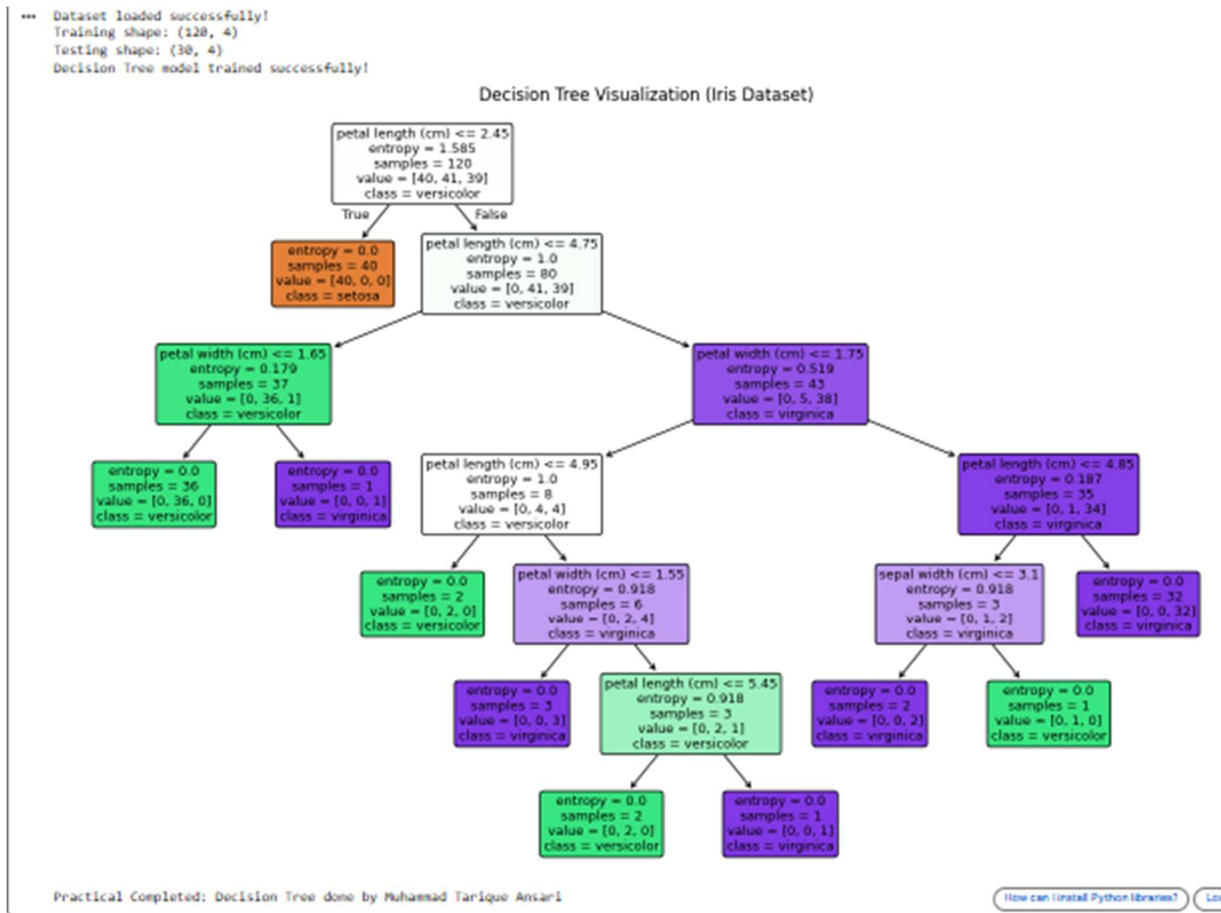
# Create a bar plot to visualize the feature
plt.figure(figsize=(10, 6))

plt.bar(iris.feature_names, feature_importances)
plt.title("Decision Tree Feature Importance")
plt.xlabel("Features")
plt.ylabel("Importance")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
```

```
plt.show()  
plt.close()
```



```
from sklearn.tree import  
DecisionTreeClassifier  
# Create and train the decision tree model  
dt_clf = DecisionTreeClassifier()  
dt_clf.fit(X, y)  
# Plot the tree  
plot_tree(dt_clf,  
           feature_names=iris.feature_names,  
           class_names=iris.target_names,  
           filled=True)
```



```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Initiate a RandomForestClassifier object and fit it to the training data
rf_clf = RandomForestClassifier(n_estimators=100, random_state=42)
rf_clf.fit(X_train, y_train)

# Predict on the test set y_pred_rf
y_pred_rf = rf_clf.predict(X_test)

# Evaluate the model
accuracy_rf = accuracy_score(y_test, y_pred_rf)
precision_rf = precision_score(y_test, y_pred_rf, average='weighted')
recall_rf = recall_score(y_test, y_pred_rf, average='weighted')
f1_rf = f1_score(y_test, y_pred_rf, average='weighted')

print("Random Forest Classifier Performance:")
print(f"Accuracy: {accuracy_rf:.4f}")
```

```
print(f"Precision: {precision_rf:.4f}")
print(f"Recall: {recall_rf:.4f}") print(f"F1-score:
{f1_rf:.4f}")
```

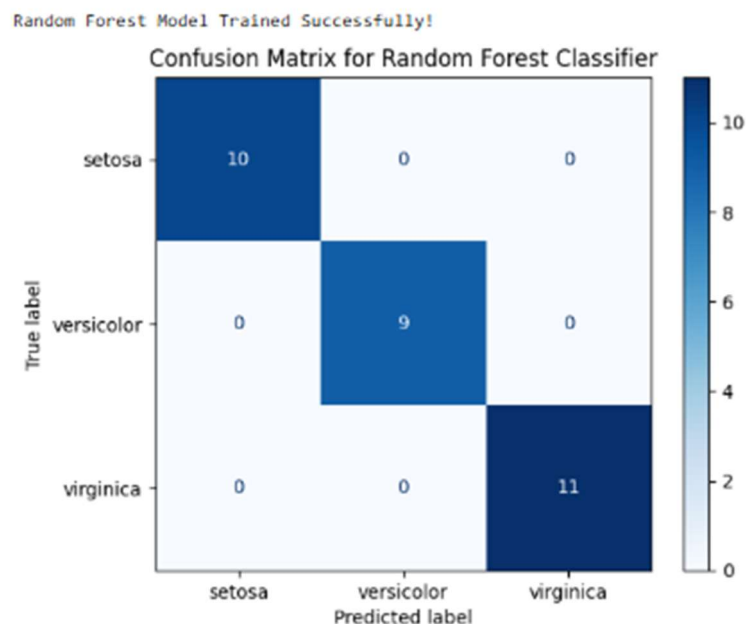
```
Random Forest Classifier Performance:
Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
F1-score: 1.0000
```

```
from sklearn.metrics import confusion_matrix, classification_report,
ConfusionMatrixDisplay import matplotlib.pyplot as plt
```

```
# Generate the confusion matrix cm =
confusion_matrix(y_test, y_pred_rf)
```

```
# Display the confusion matrix as a plot disp =
ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=iris.target_names)
disp.plot(cmap=plt.cm.Blues)
plt.title("Confusion Matrix for Random Forest Classifier") plt.show()
```

```
# Print the classification report
print("Classification Report for Random Forest Classifier:")
print(classification_report(y_test, y_pred_rf,
target_names=iris.target_names))
print("practical no 10, done by Muhammad tarique")
```



Classification Report for Random Forest Classifier:

...

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	1.00	1.00	1.00	9
virginica	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

PRACTICAL 10 COMPLETED – Done by Muhammad Tarique Ansari

Conclusion:

In this practical, a Decision Tree and a Random Forest classifier were trained on the Iris dataset and their performances were compared. The Random Forest achieved better evaluation scores than the single Decision Tree, showing that ensemble methods reduce overfitting and improve classification accuracy compared to an individual tree model.

Practical 11

Aim: Implementation of Feedforward Neural Network for Handwritten Digit Recognition using MNIST Dataset.

Theory:

A Feedforward Neural Network (FNN), also known as a Multilayer Perceptron (MLP), is one of the simplest forms of Artificial Neural Networks. In an FNN, data moves in one direction — from input to output — without looping back. Each layer consists of neurons that apply linear transformations followed by nonlinear activation functions to learn complex relationships between input and output data.

In this practical, we implement an FNN to recognize handwritten digits from the **MNIST dataset**, which contains 70,000 grayscale images of digits (0–9), each of size **28×28 pixels**. **MNIST Dataset**

1. MNIST stands for Modified National Institute of Standards and Technology dataset.
2. It contains 70,000 grayscale images of handwritten digits (0–9).
3. Each image is of size 28×28 pixels.
4. The dataset is divided into:
 - 60,000 training images
 - 10,000 testing images
5. Pixel values range from 0 (black) to 255 (white).
6. Labels represent digits 0 to 9.
7. Commonly used for handwritten digit recognition and image classification tasks.

○ The Learning Process (Training)

The network learns to recognize digits through a process called backpropagation, driven by the optimization of a loss function.

A. Forward Pass

- The input image is passed through the network, layer by layer, generating a prediction

B. Loss Calculation

- The predicted output $\hat{y}$ is compared against the true label y (the actual digit). The most common loss function for multi-class classification is Categorical Cross-Entropy:

C. Backpropagation

- The calculated loss is propagated backward through the network.
- This process computes the gradient (the rate of change of the loss with respect to each weight and bias in the network).

D. Weight Update (Optimization)

- An optimizer (e.g., Stochastic Gradient Descent or Adam) uses these gradients to adjust the network's weights and biases slightly to reduce the loss in the next iteration.

Code and Output:

```
import tensorflow as tf from tensorflow.keras.datasets import
mnist from tensorflow.keras.models import Sequential from tensorflow.keras.layers
import Dense, BatchNormalization, Dropout, Flatten, Activation import
matplotlib.pyplot as plt import numpy as np
```

```
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```



```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-
-datasets/mnist.npz
11490434/11490434 ————— 4s 0us/step
```

```
#Preprocessing and Normalize pixel values to [0,1]
x_train = x_train.astype('float32') / 255.0 x_test =
x_test.astype('float32') / 255.0
#Flatten the 28x28 images into 784-dimensional vectors
x_train = x_train.reshape(-1, 28*28) x_test =
x_test.reshape(-1, 28*28) #Convert labels to one-hot
encoded format y_train =
tf.keras.utils.to_categorical(y_train, 10) y_test =
tf.keras.utils.to_categorical(y_test, 10)
#Model Architecture model
= Sequential([

    # Input layer and 1st hidden layer
    Dense(256, input_shape=(784,)),
    BatchNormalization(),
    Activation('relu'),
    Dropout(0.3),

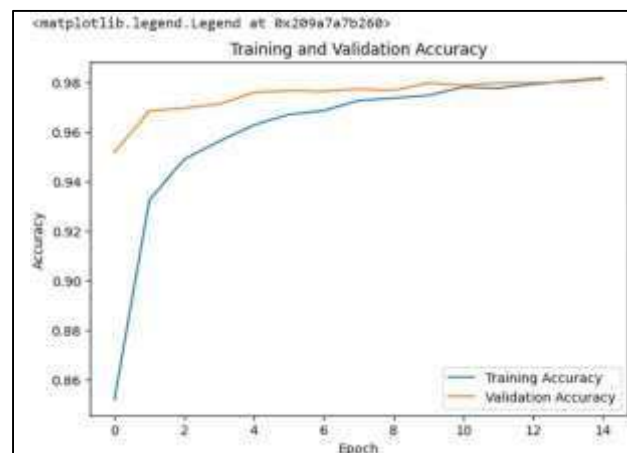
    # 2nd hidden layer
    Dense(128),
    BatchNormalization(),
    Activation('relu'),
    Dropout(0.3),
    # 3rd hidden layer
    Dense(64),
    BatchNormalization(),
    Activation('relu'),
    Dropout(0.2),
    # Output layer
    Dense(10, activation='softmax')

])
#Training using "adam" optimizer
model.compile(optimizer='adam', loss='categorical_crossentropy',
              metrics=['accuracy'])
              history = model.fit(x_train, y_train, epochs=15, batch_size=128,

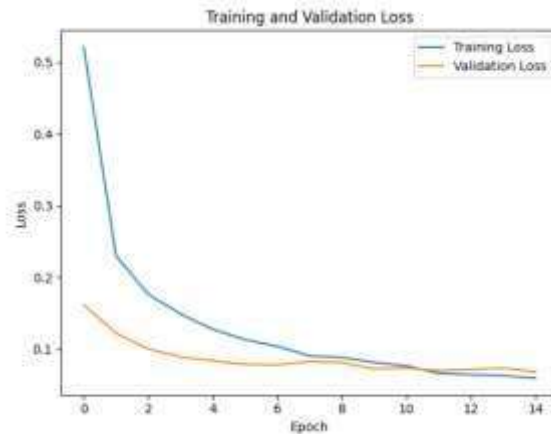
                                  validation_split=0.2, verbose=1)
```

```
Epoch 1/15
375/375 - 16s 20ms/step - accuracy: 0.8522 - loss: 0.5245 - val_accuracy: 0.9517 - val_loss: 0.1584
Epoch 2/15
375/375 - 5s 14ms/step - accuracy: 0.9324 - loss: 0.2289 - val_accuracy: 0.9688 - val_loss: 0.1076
Epoch 3/15
375/375 - 6s 15ms/step - accuracy: 0.9489 - loss: 0.1746 - val_accuracy: 0.9695 - val_loss: 0.0968
Epoch 4/15
375/375 - 5s 14ms/step - accuracy: 0.9561 - loss: 0.1462 - val_accuracy: 0.9711 - val_loss: 0.0971
Epoch 5/15
375/375 - 6s 15ms/step - accuracy: 0.9627 - loss: 0.1258 - val_accuracy: 0.9758 - val_loss: 0.0818
Epoch 6/15
375/375 - 5s 14ms/step - accuracy: 0.9669 - loss: 0.1110 - val_accuracy: 0.9765 - val_loss: 0.0779
```

```
#Training vs. validation accuracy plt.figure(figsize=(16, 5))
plt.subplot(1, 2, 1) plt.plot(history.history['accuracy'],
label='Training Accuracy') plt.plot(history.history['val_accuracy'],
label='Validation Accuracy') plt.title('Training and Validation
Accuracy') plt.xlabel('Epoch') plt.ylabel('Accuracy') plt.legend()
```



```
#Training vs. validation loss plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 2) plt.plot(history.history['loss'],
label='Training Loss') plt.plot(history.history['val_loss'],
label='Validation Loss') plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss') plt.legend()
plt.tight_layout() plt.show()
```



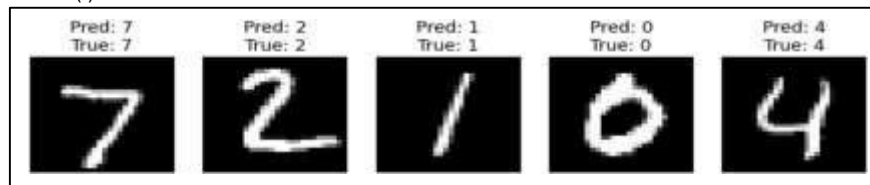
```
#Evaluation
test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
print(f"\nTest Accuracy: {test_accuracy:.4f}") print(f"Test Loss: {test_loss:.4f}")
```

```
Test Accuracy: 0.9811
Test Loss: 0.0624
```

```
y_pred = model.predict(x_test) y_pred_classes =
np.argmax(y_pred, axis=1) y_test_classes =
np.argmax(y_test, axis=1)
```

```
313/313 ————— 1s 2ms/step
```

```
#Display a few example predictions
plt.figure(figsize=(10, 2)) for i
in range(5):
    plt.subplot(1, 5, i+1)
    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')
    plt.title(f'Pred: {y_pred_classes[i]}\nTrue: {y_test_classes[i]}')
    plt.axis('off')
plt.show()
```



Conclusion:

The MNIST dataset was used to train a Feedforward Neural Network for handwritten digit recognition. The model was built, trained, and evaluated to achieve high accuracy in classifying digits 0–9.

Practical 12

Aim: Implementation of Bayesian Optimization.

Theory:

- Bayesian Optimization is a powerful optimization technique that leverages the principles of Bayesian inference to find the minimum (or maximum) of an objective function efficiently. Unlike traditional optimization methods that require extensive evaluations, Bayesian Optimization is particularly effective when dealing with expensive, noisy, or black-box functions. Bayesian Optimization is a strategy for optimizing expensive-to-evaluate functions.
- It operates by building a probabilistic model of the objective function and using this model to select the most promising points to evaluate next. This approach is particularly useful in scenarios where the objective function is unknown, noisy, or costly to evaluate, as it aims to minimize the number of evaluations required to find the optimal solution.
- **The optimization process involves two main components:**
 1. **Surrogate Model:** A probabilistic model (often a Gaussian Process) that approximates the objective function.
 2. **Acquisition Function:** A utility function that guides the selection of the next point to evaluate based on the surrogate model. Bayesian optimization effectively combines statistical modeling and decision-making strategies to optimize complex, costly functions. Here's a more detailed explanation of the process, including key formulas.
- **Key Concepts in Bayesian Optimization**
 1. **Gaussian Process (GP):** A Gaussian Process is a non-parametric model that defines a distribution over functions. In Bayesian Optimization, GPs are often used as the surrogate model because they provide not only an estimate of the objective function but also a measure of uncertainty.
 2. **Acquisition Functions:**
 - **Expected Improvement (EI):** A popular acquisition function that selects points where the expected improvement over the current best solution is maximized.
 - **Probability of Improvement (PI):** Chooses points with the highest probability of improving the current best solution.
 - **Upper Confidence Bound (UCB):** Balances exploration and exploitation by selecting points based on a confidence interval around the GP prediction.
 3. **Exploration vs. Exploitation:** Exploration involves searching in areas of the search space with high uncertainty, while exploitation focuses on areas where the surrogate model predicts good outcomes. The acquisition function manages this trade-off to efficiently find the optimum.

Code and Output

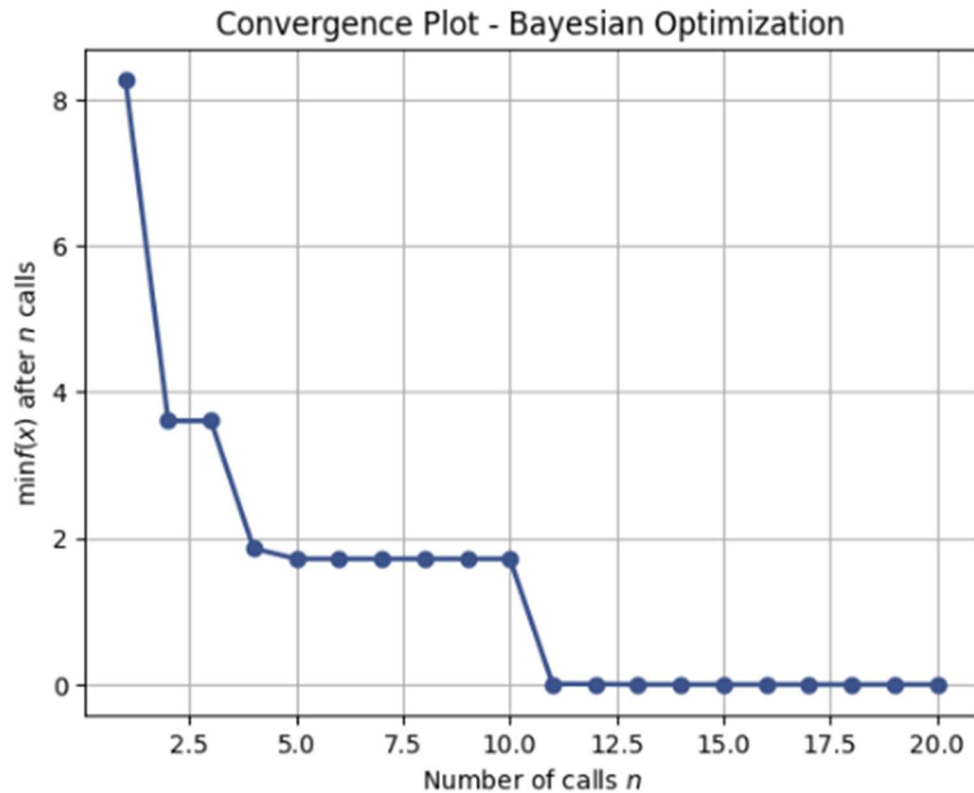
```
!pip install scikit-optimize
```

```
Requirement already satisfied: scikit-optimize in c:\users\alfiya\appdata\local\progra
Requirement already satisfied: joblib>=0.11 in c:\users\alfiya\appdata\local\programs\
Requirement already satisfied: pyaml>=16.9 in c:\users\alfiya\appdata\local\programs\p
Requirement already satisfied: numpy>=1.20.3 in c:\users\alfiya\appdata\local\programs
5)
Requirement already satisfied: scipy>=1.1.0 in c:\users\alfiya\appdata\local\programs\
3)
Requirement already satisfied: scikit-learn>=1.0.0 in c:\users\alfiya\appdata\local\pr
(1.7.2)
Requirement already satisfied: packaging>=21.3 in c:\users\alfiya\appdata\local\progra
0)
Requirement already satisfied: PyYAML in c:\users\alfiya\appdata\local\programs\python
(6.0.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\alfiya\appdata\local\p
0.0->scikit-optimize) (3.6.0)

[notice] A new release of pip available: 22.3.1 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
import numpy as np
from skopt import gp_minimize
from skopt.space import Real, Integer
from skopt.plots import plot_convergence
# Define the objective function to minimize
def objective_function(x):
    return (x[0] - 2) ** 2 + (x[1] - 3) ** 2
# Define the search space
space = [Real(0.0, 5.0, name='x1'), # Continuous space for x1
         Real(0.0, 5.0, name='x2')] # Continuous space for x2
# Perform Bayesian Optimization
result = gp_minimize(objective_function,
# The function to minimize                space,
# The search space                        n_calls=20,
# The number of evaluations                random_state=42)
# Random state for reproducibility
# Print the best parameters and the corresponding minimum value
print("Best parameters: x1 = {:.4f}, x2 = {:.4f}".format(result.x[0],
result.x[1]))
print("Minimum value: {:.4f}".format(result.fun))
# Plot convergence plot_convergence(result)
print("Practical 12 done by Muhammad Tarique")
```

```
Best parameters: x1 = 2.0015, x2 = 3.0024  
... Minimum value: 0.0000
```



Practical 12 done Muhammad Tarique

Conclusion:

In this practical, Bayesian Optimization using `gp_minimize` from the `scikit-optimize` library was successfully implemented to minimize a simple mathematical function. After installing the required package (scikit-optimize), the objective function and search space were defined, and the optimizer efficiently explored the parameter space. The algorithm identified the best values of x_1 and x_2 that produced the minimum output of the function. The convergence plot confirmed that the optimization process gradually improved over the given iterations. Overall, this practical demonstrates how Bayesian Optimization can find optimal parameter values with fewer evaluations compared to traditional search methods.